Smart Sorting Algorithm Performance Analyzer – Benchmarking and Recommending the Fastest Sorting Algorithm using Python

Design and Analysis of Algorithms (DAA) Mini Project

A PROJECT REPORT

Submitted by

Bhumi Jindal (26MCD10071)

Submitted to

Ms. Divyanshi Sharma

in partial fulfillment for the award of the degree of

Master of Computer Applications

IN

Data Science

2026-28

Chandigarh University

2026-28

BONAFIDE CERTIFICATE

Certified that this project report **"Smart Sorting Algorithm Performance Analyzer – Benchmarking and Recommending the Fastest Sorting Algorithm using Python"** is the bonafide work of **"Bhumi Jindal (26MCD10071)"** who carried out the project work under my/our supervision. The content and findings presented herein are authentic and reflect the original work conducted during the course of this project.

SIGNATURE	**SIGNATURE**
Name: Bhumi Jindal	Name: Ms. Divyanshi Sharma
UID: 26MCD10071	Designation: Asst. Professor
Department: MCD - 2/A	SUBJECT CODE: E20679

Submitted for the project viva-voce examination held on **____ / ____ / 2026**

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all the individuals who have contributed to the success of this project. Implementing six classical sorting algorithms, validating them and measuring their running time fairly presented several challenges, and I am immensely grateful for the support and guidance I received along the way.

I extend my sincere appreciation to my project supervisor, Ms. Divyanshi Sharma, for the invaluable support and encouragement throughout the duration of this project. I also owe a special debt of gratitude for the guidance, expertise and motivation that were instrumental in achieving the goals of this work. I would also like to thank all the faculty members of the University Institute of Computing for their valuable suggestions and feedback.

A heartfelt thank you goes to my parents for their unwavering support and encouragement. I am grateful to all those who, directly or indirectly, contributed to this project.

Bhumi Jindal
UID: 26MCD10071

TABLE OF CONTENTS

S. No.	TOPICS	Page No.
–	**Abstract**	5
1.	**Aim**	5
2.	**Objective / Problem Definition** • Problem Definition • Objectives	5
3.	**Programming Languages and Tools Used**	6
4.	**Block Diagram / Design Flow / Flowchart** • Block Diagram • Flowchart	6
5.	**Algorithm / Pseudo Code** • Complexity of the Six Algorithms • Pseudo Code	7–8
6.	**Implementation**	9
7.	**Output** • Interface • Correctness Validation • Benchmark Result and Recommendation • Analysis of the Result	10
8.	**Conclusion**	11
9.	**Future Framework**	12
10.	**Learning Outcomes**	12
11.	**Application Code of Project**	12–14
12.	**References**	15
13.	**GitHub Repository**	16

LIST OF FIGURES

Figure 1: Block diagram of the Smart Sorting Algorithm Performance Analyzer.....6
Figure 2: Flowchart of the recommendation process.....7
Figure 3: Interface of the analyzer.....10
Figure 4: Input pattern options.....10
Figure 5: Correctness validation of the six algorithms.....10
Figure 6: Benchmark result, recommendation and comparison chart.....11
Figure 7: GitHub repository of the project.....16

ABSTRACT

Sorting is one of the most fundamental operations in computer science, and many algorithms exist for it. Textbooks describe their theoretical complexity, but a student or developer who must choose one for a real input often cannot see how large the practical difference is. This project, the **Smart Sorting Algorithm Performance Analyzer**, is a Python-based DAA mini project that benchmarks six classical sorting algorithms – Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort and Heap Sort – and recommends the fastest one for a selected input size and input pattern.

The user chooses an input size (100 to 4,000 elements) with a slider and an input pattern (random, already sorted, reverse sorted or nearly sorted) from a drop-down list, and clicks the **Recommend Best Algorithm** button. The program generates one dataset, gives every algorithm a copy of it, verifies each output against Python's built-in sorted() function, measures the average execution time of three runs with time.perf_counter(), and then displays the recommended algorithm, a comparison table and a bar chart. The detailed result is also saved to a CSV file.

In the demonstration run with 2,200 random elements, Quick Sort was the fastest at 5.4778 ms, followed by Merge Sort (5.9467 ms) and Heap Sort (6.7236 ms), whereas Selection Sort (154.1329 ms), Insertion Sort (183.7848 ms) and Bubble Sort (610.9646 ms) were far slower. All six algorithms passed the correctness test. The project is implemented in Google Colab / Jupyter Notebook and is published on GitHub.

Keywords: Sorting Algorithms, Time Complexity, Benchmarking, Performance Analysis, Python, ipywidgets, Matplotlib, pandas.

1. AIM

To design and implement a **Smart Sorting Algorithm Performance Analyzer** in Python that compares six classical sorting algorithms – Bubble, Selection, Insertion, Merge, Quick and Heap Sort – by experiment, and that recommends the fastest algorithm for a user-selected input size and input pattern, so that the theoretical time complexities learned in Design and Analysis of Algorithms can be seen as measured evidence.

2. OBJECTIVE / PROBLEM DEFINITION

2.1 Problem Definition

Given a list of n integers, several sorting algorithms can arrange it in ascending order, but they differ greatly in running time. Algorithms with $O(n^2)$ time (Bubble, Selection, Insertion) look similar on paper to those with O(n log n) time (Merge, Quick, Heap) for small n, and the textbook tables do not show how the winner changes with the **input size** and the **initial order** of the data (random, sorted, reverse or nearly sorted). The problem is to build a tool that, for any chosen size and pattern, **runs all six algorithms on the same data, checks that their output is correct, measures their execution time fairly, and recommends the algorithm that is fastest in that experiment.**

2.2 Objectives

- To implement Bubble, Selection, Insertion, Merge, Quick and Heap Sort in Python without using the built-in sort for the algorithms themselves.
- To generate four types of test data – random, already sorted, reverse sorted and nearly sorted – of a user-selected size.
- To validate every algorithm against Python's sorted() function, using duplicates, negative numbers and mixed ordering, before any timing is trusted.
- To measure the average execution time of each algorithm over repeated runs on identical data using time.perf_counter().
- To recommend the fastest observed algorithm and present the results as a table and a bar chart.

- To provide a simple button-driven interface (slider, drop-down, button) so that no manual input() prompts are needed.
- To relate the measured results to the theoretical best, average and worst-case complexities.
- To publish the notebook, screenshots and documentation on GitHub.

3. PROGRAMMING LANGUAGES AND TOOLS USED

The whole project is written in **Python 3**. Python was chosen because its syntax is close to pseudo code, which keeps the algorithms readable, and because it has ready-made libraries for tables, charts and interactive widgets.

Technology	Purpose in the project
Python 3	Core programming language used for the data generator, the six sorting algorithms, validation and benchmarking.
random (module)	Creates random test data; random.seed(42) is set so that the experiments are repeatable.
time (module)	time.perf_counter() gives a high-resolution timer for measuring execution time.
pandas	Stores results in a DataFrame, sorts them by time, formats the table and writes the CSV file.
Matplotlib	Draws the bar chart comparing the average time of the algorithms.
ipywidgets	Provides the interactive slider, drop-down list, button and output area.
IPython.display	Displays tables, widgets and the download link for the CSV file.
Google Colab / Jupyter Notebook	Interactive environment that combines code, explanations and output in one notebook.
Git and GitHub	Version control and public hosting of the notebook, README and screenshots.

4. BLOCK DIAGRAM / DESIGN FLOW / FLOWCHART

The design follows a simple pipeline: user settings → generate input → validate the algorithms → measure repeated run times → recommend the fastest observed algorithm → show the comparison table and chart. Each stage is a separate function, which keeps the program easy to test and extend.

4.1 Block Diagram

Figure 1 shows the main blocks of the system. The user interface collects the settings, the input generator prepares one dataset, the validation block confirms that every algorithm sorts correctly, the benchmark engine times the six algorithms, the result analyzer ranks them, and the output block presents the recommendation.

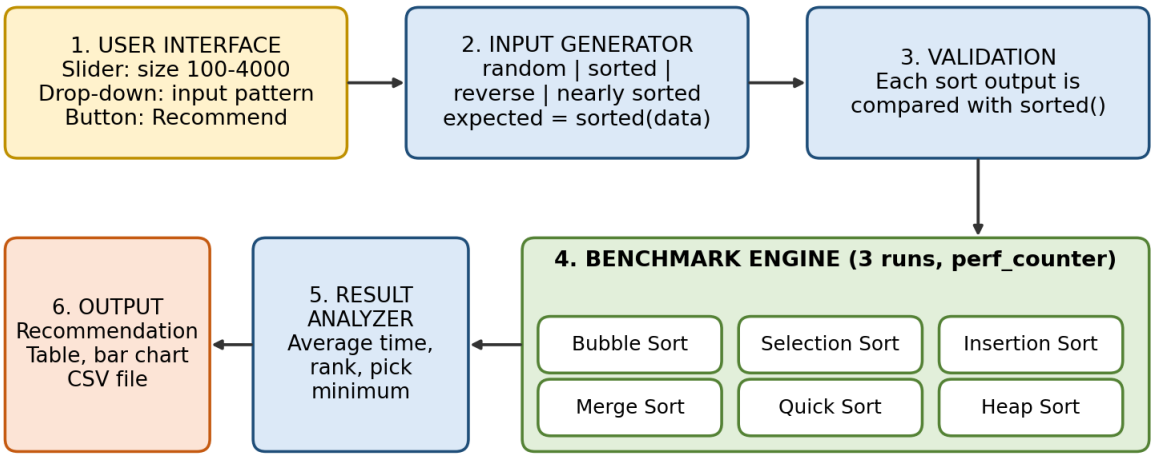


Figure 1: Block diagram of the Smart Sorting Algorithm Performance Analyzer

4.2 Flowchart

The flowchart in Figure 2 shows the steps executed when the user clicks the Recommend Best Algorithm button. The loop repeats until all six algorithms have been measured; if any algorithm returns an incorrectly sorted list, the program stops with an error rather than reporting a misleading time.

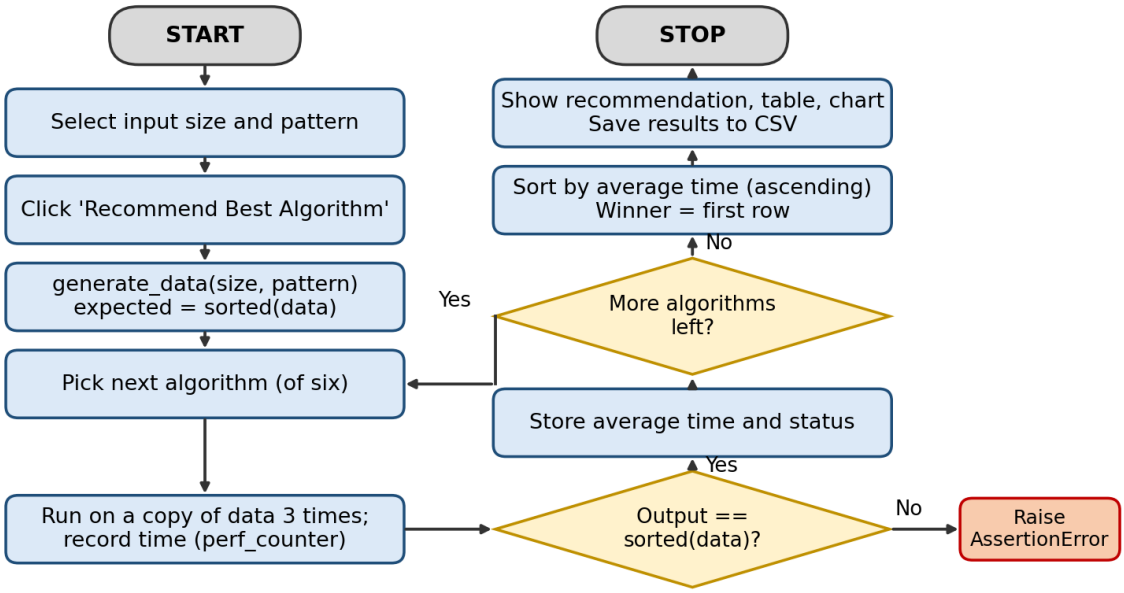


Figure 2: Flowchart of the recommendation process

5. ALGORITHM / PSEUDO CODE

5.1 Complexity of the Six Algorithms

The theoretical complexity explains how the running time grows with the input size n; the measured results in Chapter 7 provide empirical evidence for the selected input and the computer used.

Algorithm	Best	Average	Worst	Extra space
Bubble Sort (optimized)	O(n)	O(n²)	O(n²)	O(1)
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort (this implementation)	O(n log n)	O(n log n)	O(n²)	O(n) in this list-based version
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1) auxiliary

5.2 Pseudo Code

The pseudo code below follows the Python implementation. Every sorting function first makes a copy of the input so that the original data is never changed.

Algorithm 1: generate_data(size, type)

```
IF size is not a positive integer THEN raise error
IF type = random         THEN RETURN size random integers in [1, 100000]
IF type = sorted         THEN RETURN [0, 1, ..., size-1]
IF type = reverse        THEN RETURN [size, size-1, ..., 1]
IF type = nearly_sorted THEN
    data <- [0, 1, ..., size-1]
    REPEAT max(1, size/50) times: swap two random positions
    RETURN data
ELSE raise error
```

Algorithm 2: Bubble Sort

```
a <- copy of arr ;  n <- length(a)
FOR i <- 0 TO n-1
    swapped <- false
    FOR j <- 0 TO n-i-2
        IF a[j] > a[j+1] THEN swap a[j], a[j+1] ; swapped <- true
    IF swapped = false THEN BREAK
RETURN a
```

Algorithm 3: Selection Sort

```
a <- copy of arr ;  n <- length(a)
FOR i <- 0 TO n-1
    min_index <- i
    FOR j <- i+1 TO n-1
        IF a[j] < a[min_index] THEN min_index <- j
    swap a[i], a[min_index]
RETURN a
```

Algorithm 4: Insertion Sort

```
a <- copy of arr
FOR i <- 1 TO length(a)-1
    key <- a[i] ;  j <- i-1
    WHILE j >= 0 AND a[j] > key
        a[j+1] <- a[j] ;  j <- j-1
    a[j+1] <- key
RETURN a
```

Algorithm 5: Merge Sort

```
IF length(arr) <= 1 THEN RETURN copy of arr
mid <- length(arr) / 2
left  <- MergeSort(arr[0 .. mid-1])
right <- MergeSort(arr[mid .. end])
merged <- empty list ;  i <- 0 ;  j <- 0
WHILE i < length(left) AND j < length(right)
    IF left[i] <= right[j] THEN append left[i] ; i <- i+1
    ELSE append right[j] ; j <- j+1
append the remaining elements of left and right
RETURN merged
```

Algorithm 6: Quick Sort (three-way, list based)

```
IF length(arr) <= 1 THEN RETURN copy of arr
pivot <- arr[length(arr) / 2]
lower <- all x in arr with x < pivot
equal <- all x in arr with x = pivot
upper <- all x in arr with x > pivot
RETURN QuickSort(lower) + equal + QuickSort(upper)
```

Algorithm 7: Heap Sort

```
a <- copy of arr ;  n <- length(a)
FOR start <- n/2 - 1 DOWN TO 0 : SiftDown(a, n, start)   // build max-heap
FOR end <- n-1 DOWN TO 1
    swap a[0], a[end]
    SiftDown(a, end, 0)
RETURN a

SiftDown(a, end, root):
    LOOP: child <- 2*root + 1 ; IF child >= end THEN RETURN
          IF child+1 < end AND a[child] < a[child+1] THEN child <- child+1
          IF a[root] >= a[child] THEN RETURN
          swap a[root], a[child] ; root <- child
```

Algorithm 8: measure_average_time(sort_function, data, runs = 3)

```
timings <- empty list
REPEAT runs times
    start <- perf_counter()
    result <- sort_function(data)
    append (perf_counter() - start) to timings
    IF result != sorted(data) THEN raise AssertionError
RETURN sum(timings) / runs
```

Algorithm 9: Recommend Best Algorithm (main procedure)

```
size <- slider value ;  type <- drop-down value
data <- generate_data(size, type) ;  expected <- sorted(data)
FOR EACH (name, sort_function) IN the six algorithms
    t <- measure_average_time(sort_function, data, 3)
    correct <- (sort_function(data) = expected)
    add row (name, t in seconds, t in milliseconds, correct) to results
sort results by time in ascending order
winner <- first row of results
DISPLAY winner, results table and bar chart
SAVE results to sorting_recommendation_results.csv
```

6. IMPLEMENTATION

The project is implemented as a single Google Colab / Jupyter notebook that is run from top to bottom. It is divided into the following steps.

6.1 Libraries and Setup

The notebook imports random, time, pandas, matplotlib and ipywidgets (installed automatically if missing). In Google Colab the custom widget manager is enabled, and random.seed(42) makes the generated data repeatable.

6.2 Test Data Generation

The function generate_data(size, data_type) returns a list of integers. Random data uses values from 1 to 100,000; sorted data is 0 to size-1; reverse data counts down from size to 1; nearly sorted data starts sorted and then performs size/50 random swaps. Invalid sizes or unknown types raise a ValueError.

6.3 Sorting Algorithms

All six algorithms are written by hand and each returns a **sorted copy** of the list, leaving the source data unchanged, which keeps the comparison fair. Bubble Sort stops early when a pass makes no swap. Quick Sort uses the middle element as pivot and splits the list into smaller, equal and larger parts, so duplicate values do not cause deep recursion. Heap Sort builds a max-heap in place and repeatedly moves the largest element to the end. All algorithms are stored in a dictionary named algorithms, so a new algorithm can be added with one line.

6.4 Correctness Validation

Timing an incorrect program would make the comparison meaningless. Therefore every algorithm is first tested on the list [23, -4, 12, 12, 0, 91, 5, -4], which contains duplicates, negative values and mixed ordering, and its output is compared with sorted(). A failure raises an AssertionError. A further check is made during benchmarking, where each run is compared with the expected output.

6.5 Benchmarking

The function measure_average_time() runs a sort function three times on the same data, records each duration with time.perf_counter() and returns the average. The results are placed in a pandas DataFrame, sorted by time, and the first row is the recommended algorithm. The program states clearly that the result is empirical – valid for the selected input and the current session – and not a universal winner.

6.6 Interactive Interface

The interface is built with ipywidgets: an IntSlider for the input size (100 to 4,000 in steps of 100, default 1,000), a Dropdown for the input pattern (Random values, Already sorted, Reverse sorted, Nearly sorted), a green **Recommend Best Algorithm** button and an Output area. Clicking the button calls run_recommendation(), which generates the data, runs the benchmark, prints the recommendation, shows the table and bar chart, and writes the CSV file with a download link.

6.7 How the Recommendation Works

- The input generator creates **one dataset** matching the selected size and pattern.
- Every algorithm receives that same dataset, and its function sorts a **copy**, so one algorithm does not change another algorithm's input.
- Each algorithm is timed **three times** using time.perf_counter(); the average is used for ranking.
- The algorithm with the **lowest measured average time** is recommended for that experiment.
- A **correctness check** ensures the returned list matches Python's sorted() output.
- The detailed result of the latest run is saved to **sorting_recommendation_results.csv** in the Colab working directory, and a download link is shown.

Important limitation: micro-benchmarks are affected by the computer, the Python runtime and small timing fluctuations. Big-O complexity and the observed measurements should be considered together; the winner is not claimed to be universally best. **Demo tip:** start with a small size such as 1,000 elements, try Random and Already sorted inputs, and explain why the winning algorithm can differ by input pattern. Bubble Sort and Selection Sort become slow as the input grows because they have quadratic time complexity.

7. OUTPUT

7.1 Interface

Figure 3 shows the interface of the analyzer. The user sets the input size with the slider and presses the button; a note advises starting with 1,000 elements for a fast demonstration because larger inputs take longer. Figure 4 shows the four input patterns that can be selected.

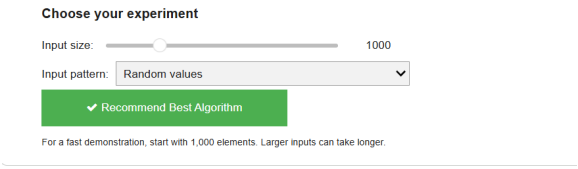


Figure 3: Interface of the analyzer

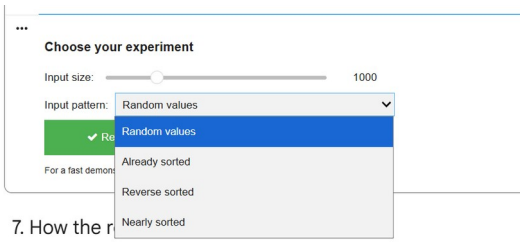


Figure 4: Input pattern options

7.2 Correctness Validation

Figure 5 shows the correctness test on the list [23, -4, 12, 12, 0, 91, 5, -4], which has duplicates and negative values. All six algorithms returned True and the notebook printed "All algorithms passed the correctness test."

	Algorithm	Correct output
0	Bubble Sort	True
1	Selection Sort	True
2	Insertion Sort	True
3	Merge Sort	True
4	Quick Sort	True
5	Heap Sort	True

All algorithms passed the correctness test.

Figure 5: Correctness validation of the six algorithms

7.3 Benchmark Result and Recommendation

Figure 6 shows the output for a dataset of 2,200 random elements with three runs per algorithm. The system recommends **Quick Sort** as the best observed algorithm (average 5.4778 ms), and every algorithm returned a correctly sorted list. Table 1 lists the measured times.

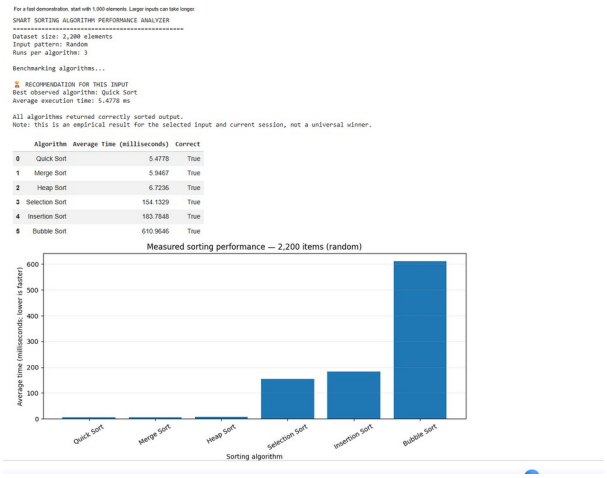


Figure 6: Benchmark result, recommendation and comparison chart

Rank	Algorithm	Avg (ms)	Correct
1	Quick Sort	5.4778	True
2	Merge Sort	5.9467	True
3	Heap Sort	6.7236	True
4	Selection Sort	154.1329	True
5	Insertion Sort	183.7848	True
6	Bubble Sort	610.9646	True

Table 1: Measured result, 2,200 random elements

7.4 Analysis of the Result

- **O(n log n) algorithms win clearly.** Quick, Merge and Heap Sort finished in about 5–7 ms, while the three O(n^2) algorithms needed 154–611 ms for the same data.
- **Quick Sort was the fastest in this run**, about 8.6% faster than Merge Sort and 22.7% faster than Heap Sort. These three are close to each other, so a different computer or run could change their order.
- **Bubble Sort was the slowest**, about 111 times slower than Quick Sort, because it makes the largest number of comparisons and swaps on random data. Insertion Sort was about 33.5 times and Selection Sort about 28 times slower than Quick Sort.
- **Input pattern matters.** By theory, Bubble Sort (optimized) and Insertion Sort fall to O(n) on already sorted data, while Selection Sort stays O(n^2); the interface lets the user check this by choosing the Already sorted or Nearly sorted pattern.
- **The result is empirical.** Timings depend on the computer, the Python runtime and small fluctuations, so the recommendation applies to the selected experiment and should be read together with Big-O complexity.

8. CONCLUSION

The Smart Sorting Algorithm Performance Analyzer combines algorithm implementation, correctness testing, experimental analysis, visualisation and a button-driven recommendation interface in one project. Six classical sorting algorithms were implemented, verified against Python's sorted() and benchmarked on identical data. In the demonstration with 2,200 random elements, Quick Sort was recommended with an average time of 5.4778 ms, while Bubble Sort took 610.9646 ms.

The experiment confirms the theory taught in Design and Analysis of Algorithms: algorithms with O(n log n) time scale far better than those with O(n^2) time as the input grows, and the input order influences the performance of each algorithm. The project also shows why measured results must be reported carefully – the winner is "best observed" for one input, one computer and one session, not a universal champion.

9. FUTURE FRAMEWORK

- **More algorithms:** add Shell Sort, Counting Sort, Radix Sort, Tim Sort and Python's built-in sorted() for comparison.
- **Better measurement:** use more runs and report the median and standard deviation; also count comparisons and swaps, and measure memory use.
- **Larger inputs and scaling curves:** allow sizes beyond 4,000 for the O(n log n) algorithms, use a logarithmic axis, and plot time against size to show growth curves.
- **Custom and real data:** let the user upload a CSV file or type their own list, and support strings and floating-point numbers.
- **Web application:** convert the notebook into a Streamlit or Flask web app that runs without Colab.
- **Smart recommendation model:** train a machine-learning model on the collected results to predict the best algorithm from the size and pattern of the data without running all six.
- **Hybrid algorithms:** add Introsort or an Insertion-Sort cut-off in Merge/Quick Sort, and compare them with the pure versions.
- **Reports:** export the comparison chart and table to a PDF report automatically.

10. LEARNING OUTCOMES

After completing this project, I was able to:

1. Implement six classical sorting algorithms and explain the working of each one.
2. Analyse best, average and worst-case time and space complexity using Big-O notation and relate them to measured results.
3. Design test data (random, sorted, reverse, nearly sorted) that exposes the strengths and weaknesses of each algorithm.
4. Validate algorithms for correctness before comparing their performance.
5. Measure execution time reliably with time.perf_counter() and understand why repeated runs and identical data are needed for a fair benchmark.
6. Use pandas for tabulating results and Matplotlib for visual comparison.
7. Build an interactive interface with ipywidgets and structure a program into reusable functions.
8. Present empirical findings honestly, with their limitations, and publish a project on GitHub with documentation.

APPLICATION CODE OF PROJECT

The complete Python code of the notebook is given below.

```
import random
import time
import pandas as pd
import matplotlib.pyplot as plt
try:
    import ipywidgets as widgets
except ModuleNotFoundError:
    import subprocess, sys
    subprocess.check_call([sys.executable, "-m", "pip", "install", "ipywidgets"])
    import ipywidgets as widgets
from IPython.display import display, clear_output, FileLink

# Enable ipywidgets in Google Colab when the Colab widget manager is available.
try:
    from google.colab import output as colab_output
    colab_output.enable_custom_widget_manager()
except Exception:
    pass

random.seed(42)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

print("Libraries loaded. Use the interactive controls near the end of this notebook.")
def generate_data(size, data_type="random"):
    """Return a list of integers with the requested size and input pattern."""
```

```
    if not isinstance(size, int) or size < 1:
        raise ValueError("Dataset size must be a positive integer.")

    if data_type == "random":
        return [random.randint(1, 100_000) for _ in range(size)]
    if data_type == "sorted":
        return list(range(size))
    if data_type == "reverse":
        return list(range(size, 0, -1))
    if data_type == "nearly_sorted":
        data = list(range(size))
        swaps = max(1, size // 50)
        for _ in range(swaps):
            i = random.randrange(size)
            j = random.randrange(size)
            data[i], data[j] = data[j], data[i]
        return data

    raise ValueError("Unknown data type. Choose random, sorted, reverse, or nearly_sorted.")
def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a


def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if a[j] < a[min_index]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
    return a


def insertion_sort(arr):
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a


def merge_sort(arr):
    if len(arr) <= 1:
        return arr.copy()
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged


def quick_sort(arr):
    """Three-way Quick Sort; handles duplicate values without deep duplicate recursion."""
    if len(arr) <= 1:
        return arr.copy()
    pivot = arr[len(arr) // 2]
    lower = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    upper = [x for x in arr if x > pivot]
    return quick_sort(lower) + equal + quick_sort(upper)


def heap_sort(arr):
    a = arr.copy()
    n = len(a)

    def sift_down(end, root):
```

```
        while True:
            child = 2 * root + 1
            if child >= end:
                return
            if child + 1 < end and a[child] < a[child + 1]:
                child += 1
            if a[root] >= a[child]:
                return
            a[root], a[child] = a[child], a[root]
            root = child

    for start in range(n // 2 - 1, -1, -1):
        sift_down(n, start)
    for end in range(n - 1, 0, -1):
        a[0], a[end] = a[end], a[0]
        sift_down(end, 0)
    return a


algorithms = {
    "Bubble Sort": bubble_sort,
    "Selection Sort": selection_sort,
    "Insertion Sort": insertion_sort,
    "Merge Sort": merge_sort,
    "Quick Sort": quick_sort,
    "Heap Sort": heap_sort,
}
print(f"Implemented {len(algorithms)} sorting algorithms:", ", ".join(algorithms))
```

```
test_data = [23, -4, 12, 12, 0, 91, 5, -4]
expected = sorted(test_data)
validation_rows = []
for name, sort_function in algorithms.items():
    actual = sort_function(test_data)
    passed = actual == expected
    validation_rows.append({"Algorithm": name, "Correct output": passed})
    if not passed:
        raise AssertionError(f"{name} failed the correctness test.")

validation_df = pd.DataFrame(validation_rows)
display(validation_df)
print("All algorithms passed the correctness test.")
```

```
# Measuring helper: each sort function works on a copy of the same data.
def measure_average_time(sort_function, data, runs=3):
    if runs < 1:
        raise ValueError("runs must be at least 1")
    timings = []
    for _ in range(runs):
        start = time.perf_counter()
        sorted_result = sort_function(data)
        timings.append(time.perf_counter() - start)
        if sorted_result != sorted(data):
            raise AssertionError("An algorithm returned an incorrect result during benchmarking.")
    return sum(timings) / len(timings)

# Input controls
size_slider = widgets.IntSlider(
    value=1000, min=100, max=4000, step=100,
    description="Input size:", continuous_update=False,
    style={"description_width": "initial"},
    layout=widgets.Layout(width="420px")
)
input_dropdown = widgets.Dropdown(
    options=[
        ("Random values", "random"),
        ("Already sorted", "sorted"),
        ("Reverse sorted", "reverse"),
        ("Nearly sorted", "nearly_sorted"),
    ],
    value="random", description="Input pattern:",
    style={"description_width": "initial"},
    layout=widgets.Layout(width="420px")
)
run_button = widgets.Button(
    description="Recommend Best Algorithm",
    button_style="success", icon="check",
    tooltip="Benchmark all six algorithms and recommend the fastest observed one",
    layout=widgets.Layout(width="280px", height="42px")
)
recommendation_output = widgets.Output()


def run_recommendation(_button):
    with recommendation_output:
        clear_output(wait=True)
        size = int(size_slider.value)
        input_type = input_dropdown.value
        data = generate_data(size, input_type)
        expected = sorted(data)
        rows = []
```

```
        print("SMART SORTING ALGORITHM PERFORMANCE ANALYZER")
        print("=" * 48)
        print(f"Dataset size: {size:,} elements")
        print(f"Input pattern: {input_type.replace('_', ' ').title()}")
        print("Runs per algorithm: 3")
        print("\nBenchmarking algorithms...\n")

        for name, sort_function in algorithms.items():
            elapsed = measure_average_time(sort_function, data, runs=3)
            correct = sort_function(data) == expected
            rows.append({
                "Algorithm": name,
                "Average Time (seconds)": elapsed,
                "Average Time (milliseconds)": elapsed * 1000,
                "Correct": correct,
            })

        results_df = pd.DataFrame(rows).sort_values(
            "Average Time (seconds)", ascending=True
        ).reset_index(drop=True)
        winner = results_df.iloc[0]

        print("🏆 RECOMMENDATION FOR THIS INPUT")
        print(f"Best observed algorithm: {winner['Algorithm']}")
        print(f"Average execution time: {winner['Average Time (milliseconds)']:.4f} ms")
        print("\nAll algorithms returned correctly sorted output.")
        print("Note: this is an empirical result for the selected input and current session, not a universal
winner.\n")

        display(results_df[["Algorithm", "Average Time (milliseconds)", "Correct"]].style.format({
            "Average Time (milliseconds)": "{:.4f}"
        }))

        fig, ax = plt.subplots(figsize=(10, 5))
        plot_df = results_df.sort_values("Average Time (milliseconds)", ascending=True)
        ax.bar(plot_df["Algorithm"], plot_df["Average Time (milliseconds)"])
        ax.set_title(f"Measured sorting performance — {size:,} items ({input_type.replace('_', ' ')})")
        ax.set_xlabel("Sorting algorithm")
        ax.set_ylabel("Average time (milliseconds; lower is faster)")
        ax.tick_params(axis="x", rotation=30)
        ax.grid(axis="y", alpha=0.25)
        fig.tight_layout()
        plt.show()

        # Save the most recent interactive result in the Colab working directory.
        results_df.to_csv("sorting_recommendation_results.csv", index=False)
        print("Saved detailed results to sorting_recommendation_results.csv")
        display(FileLink("sorting_recommendation_results.csv"))

run_button.on_click(run_recommendation)

controls = widgets.VBox([
    widgets.HTML("<h3 style='margin-bottom:4px'>Choose your experiment</h3>"),
    size_slider,
    input_dropdown,
    run_button,
    widgets.HTML("<small>For a fast demonstration, start with 1,000 elements. Larger inputs can take
longer.</small>"),
])
display(controls)
display(recommendation_output)
```

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
2. R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.
3. D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed., Addison-Wesley, 1998.
4. Python Software Foundation, Python documentation – time.perf_counter(), random and built-in sorted(), https://docs.python.org/3/.
5. pandas documentation, https://pandas.pydata.org/docs/.
6. Matplotlib documentation, https://matplotlib.org/stable/.
7. ipywidgets documentation, https://ipywidgets.readthedocs.io/.

GITHUB REPOSITORY

The notebook, README and output screenshots of the project are published on GitHub at: **https://github.com/jindalbhumi125-dev/smart-sorting-algorithm-performance-analyzer**

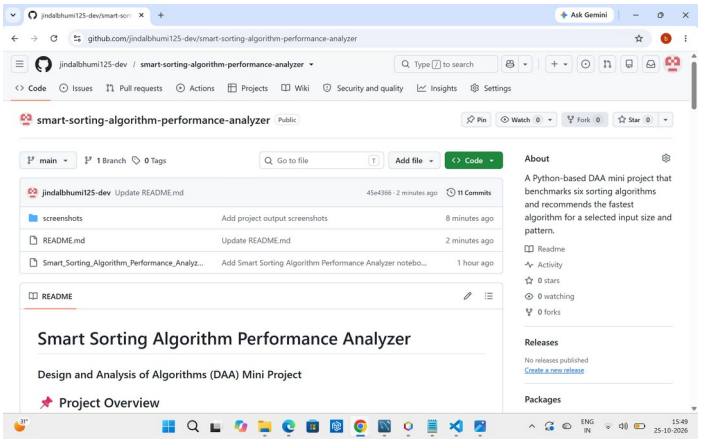


Figure 7: GitHub repository of the project